

Fundamental Concepts : How Widget Works?

Understanding Key-Value Format in Flutter

a. What is a Key-Value Pair?

- In Flutter, widget properties are defined using key-value pairs.
- Key → Represents a property (e.g., style, color).
- Value → Defines how the property behaves (e.g., Colors.blue, TextStyle).
- For example, in the Text widget:

```
Text(  
  'Hello, Flutter!',      // This is a value (text being  
displayed)  
  style: TextStyle(      // 'style' is a key  
    fontSize: 20,        // 'fontSize' is a key, and '20' is the  
value  
    color: Colors.blue,  // 'color' is a key, and 'Colors.blue'  
is the value  
  ),  
);
```

Hello, Flutter!

b. Why Use Key-Value Pairs?

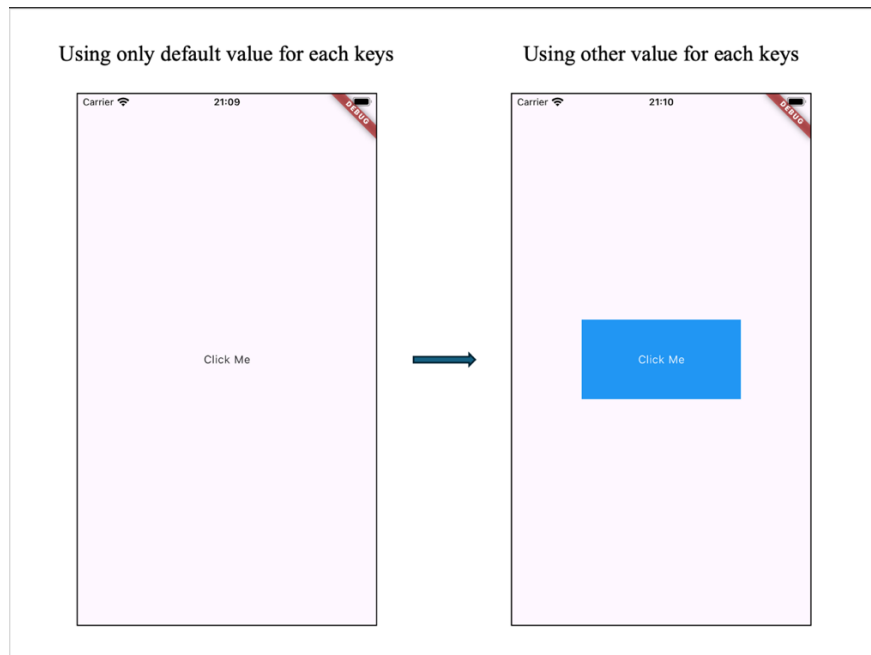
- *Customizes widget properties* → Easily control UI appearance & behavior.
- *Overrides default values* → Adjust built-in properties like color, size, and alignment.
- *Improves readability* → Named parameters make Flutter code clearer.

c. When Do You Need to Change the Key's Default Value?

- If no value is provided, Flutter applies a default value.
- You override the default when you need a different behavior.
- This is an example for changing default values :

```
Container(  
  width: 200,  
  height: 100,  
  color: Colors.blue, // This sets the background color to  
blue  
  child: Center(  
    child: Text(  
      'Click Me',  
      style: TextStyle(color: Colors.white), // Text color  
is white  
    ),  
  ),  
)
```

- Explanation:
 - By default, a Container has no background color.
 - Setting color: Colors.blue changes the background to blue.
 - Inside, Text('Click Me') is centered and displayed in white.



d. Using Key-Value Pairs in Flutter vs. JSON

- Both Flutter & JSON use key-value pairs, but they serve different purposes.
- Flutter → Defines how an app looks (UI).

```
Text (
  "Hello, World!",
  textAlign: TextAlign.center,
)
```

- JSON → Stores and transfers data.

```
{
  "text": "Hello, World!",
  "textAlign": "center"
}
```

- Key Difference :

Feature	Flutter Code	JSON
Purpose	UI strusture & logic	Data storage & transfer
Execution	Runs in an app	Used to pass data
Example	Text('Hello')	{ "text": "Hello" }

Why Widgets Are Nested in Flutter

a. What is Widget Nesting?

- Nesting widgets means placing one widget inside another.
- This allows you to combine UI elements into more complex designs.
- Every Flutter UI is built using a hierarchy of nested widgets.
- For example, a Text widget inside a Button :

```
ElevatedButton (
```

- `main()` → The starting point of the program.
- `runApp(MyApp())` → Launches the Flutter app and loads `MyApp()`.
- b. How `runApp()` Works in Flutter
 - `runApp()` loads the root widget of the app.
 - It sets up the UI hierarchy by displaying the `MyApp()` widget.
 - The entire app structure is built using widgets instead of separate UI files (like XML in Android).
- c. Comparison: `main()` in Java vs. Flutter
 - Java and Flutter both start execution with `main()`, but they handle UI differently.
 - Java (e.g., Android apps) → `main()` only starts logic; UI is built separately using XML.

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

- Flutter/Dart → `main()` immediately builds UI using widgets.
- Key Differences :

Feature	Java (Android Apps)	Flutter (Dart)
UI Handling	UI is built separately using XML	UI is built inside <code>main.dart</code> with widgets
Workflow	<code>main()</code> initializes the app, but UI is handled separately	<code>main()</code> starts the app and UI setup happens immediately
Ease of Management	UI and logic are in separate files	Everything is managed in a single file

Step 3 : Understanding `MyApp` and the Widget Tree

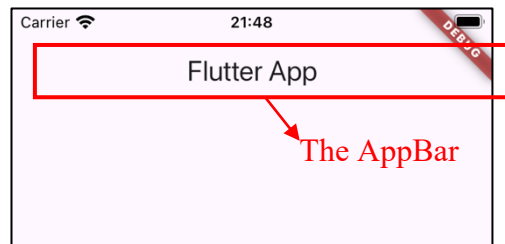
- a. What is `MyApp`?
 - After calling `runApp()`, the app loads the `MyApp` widget.
 - `MyApp` is the root widget of the Flutter application.
 - Every Flutter app is built using widgets, which form the UI.
 - This is an example for basic `MyApp` implementation :

```
class MyApp extends StatelessWidget {
    @override
    Widget build(BuildContext context) {
        return MaterialApp(
            home: Scaffold(
                appBar: AppBar(
                    title: Text('My Flutter App'),
                ),
                body: Center(
                    child: Text('Hello, world!'),
                ),
            ),
        );
    }
}
```

```

    );
  }
}

```



- Explanation:
 - MyApp → A custom widget that acts as the app's entry point.
 - StatelessWidget → MyApp does not have mutable state.
 - build() method → Defines how the UI is displayed.
 - Returns a widget tree, which is a hierarchical structure of widgets.
- b. How the Widget Tree Works
 - Flutter's UI is structured like a tree (Widget Tree).
 - Each widget can have child widgets, forming a hierarchy.
 - The widget tree defines the structure and relationships between widgets.
 - This is an example of widget tree in MyApp

```

MaterialApp(
  home: Scaffold(
    appBar: AppBar(
      title: Text('My Flutter App'),
    ),
    body: Center(
      child: Text('Hello, world!'),
    ),
  ),
);

```

- Tree structure :

```

MaterialApp
├── Scaffold
│   ├── AppBar
│   │   └── Text ('My Flutter App')
│   └── Body
│       └── Center
│           └── Text ('Hello, world!')

```

- Explanation:
 - MaterialApp → The root widget that wraps everything.
 - Scaffold → Provides the basic screen structure (AppBar, body).
 - AppBar → A bar at the top of the app (for title or actions).

- Text → Displays simple text inside the UI.
- c. Understanding Each Widget in the Tree

Widget	Function	Example
MaterialApp	The root widget that sets up themes & navigation	<code>MaterialApp(home: Scaffold(...))</code>
Scaffold	Provides the basic screen layout (AppBar, body, etc.)	<code>Scaffold(appBar: ..., body: ...)</code>
AppBar	Displays a top navigation bar	<code>AppBar(title: Text('My App'))</code>
Center	Centers its child widget inside the screen	<code>Center(child: Text('Hello'))</code>
Text	Displays simple text	<code>Text('Hello, world!')</code>

- Each widget in Flutter has a specific function and can nest inside other widgets.
- This nesting structure helps organize the UI efficiently.

Step 4 : Understanding the Class and extends

- a. What is a Class in Flutter?
- A class is like a blueprint for creating objects (in Flutter, widgets).
 - Flutter apps are built using custom widgets, which are defined as classes.
 - Each widget class defines how the UI looks and behaves.
 - This is how usually we defining a widget class in Flutter :

```
class MyApp extends StatelessWidget {
```

- Explanation:
 - class MyApp → Defines a new widget called MyApp.
 - extends StatelessWidget → MyApp is based on StatelessWidget, meaning it does not change dynamically.
- b. What Does extends Mean in Flutter?
- The extends keyword creates a subclass (child class) of another class.
 - In Flutter, extends StatelessWidget means the widget is immutable (does not change state).
 - A widget only updates when its parent widget rebuilds it.
 - This is an example of full Stateless Widget :

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text('My Flutter App')),
        body: Center(child: Text('Hello, Flutter!')),
      ),
    );
  }
}
```

- When you need a consistent top bar for your app.
- When you want to add a title or navigation buttons.
- When you need search, settings, or more options icons.
- When you want to add tabs or extra content below the AppBar.

Text Widget

- What is the Text Widget?
 - Displays text on the screen in a Flutter app.
 - Supports both plain and styled text.
 - Can be used for static messages, dynamic updates, and responsive design.
- Key Properties of the Text Widget
 - Commonly used properties:

Property	Description	Example
data	The text to display (required).	'Hello, Flutter!'
style	Customizes the text appearance.	TextStyle(fontSize: 18, color: Colors.blue)
textAlign	Aligns text (left, center, right).	TextAlign.center
maxLines	Limits the number of text lines.	maxLines: 2
overflow	Handles text overflow (ellipsis, fade).	TextOverflow.ellipsis
softWrap	Controls text wrapping.	softWrap: true

- Example: Styling Text in Flutter
 - Code Example:

```
Text (
  'Welcome to Flutter!', // The text to display
  style: TextStyle(
    fontSize: 20,           // Sets the font size
    color: Colors.green,    // Text color
    fontWeight: FontWeight.bold, // Makes the text bold
  ),
  textAlign: TextAlign.center, // Centers the text
  maxLines: 2,               // Limits to 2 lines
  overflow: TextOverflow.ellipsis, // Adds "..." if the
text is too long
)
```

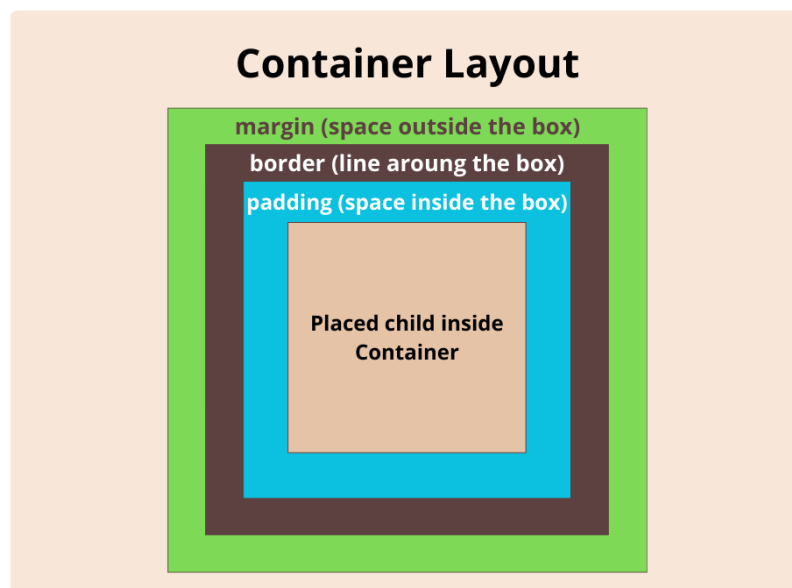
- Explanation :
 - 'Welcome to Flutter!' → The text to display.
 - style → Defines size, color, weight, etc.
 - textAlign: TextAlign.center → Centers the text.
 - maxLines: 2 → Limits text to 2 lines.
 - overflow: TextOverflow.ellipsis → Shows "..." if text is too long.
- d. Why Use the Text Widget?
 - Advantages of using the Text widget:
 - Easy to use for both static & dynamic text.
 - Highly customizable (alignment, wrapping, overflow, etc.).

- Flexible for any layout or UI design.

Container Widget

- What is the Container Widget?
 - A customizable “box” used to style, position, and layout elements in an app.
 - Often used to wrap other widgets and apply spacing, colors, or decorations.
 - One of the most flexible and commonly used widgets in Flutter.
- Key Features of Container
 - Commonly used properties:

Property	Description	Example
margin	Adds space outside the container.	margin: EdgeInsets.all(10)
padding	Adds space inside the container.	padding: EdgeInsets.all(15)
width & height	Sets the size of the container.	width: 200, height: 100
color	Sets the background color.	color: Colors.blue
decoration	Adds styling like borders, gradients, and shadows.	decoration: BoxDecoration(...)



- Customizing Container with BoxDecoration
 - Additional styling options with BoxDecoration:
 - color → Sets the background color.
 - border → Adds a border around the container.
 - borderRadius → Rounds the corners of the container.
 - boxShadow → Adds a shadow effect.
 - gradient → Creates a gradient background.
- Example: Styled Container in Flutter
 - Code Example:

```
Container(
  width: 200, // Width of the container
```